Introduction
1. Data Loading & Preprocessing
2. Volatility Analysis
3. Rolling Window
4. Price Simulation
5. Simulated vs Real Prices
6. Quantile Bands
7. Model Validation: RMSE
Conclusion

# VOLATILITY MODELING AND FORECASTING WITH GARCH MODELS

### Analyzing and Predicting Google Stock Prices

**Luca Celio**

**2025-03-17**

## Introduction

This report provides a comprehensive analysis of the volatility dynamics of Google stock prices using a range of GARCH models. My analysis aims to:

- Estimate historical volatility trends and assess their evolution over time.
- Fit several GARCH model variants, including standard GARCH, asymmetric (GJR-GARCH) and exponential (eGARCH) models, with both normal and skewed Student-t distributions, to capture market behavior and account for non-linear effects.
- Forecast future volatility for January-February 2025 using a rolling window approach.
- Simulate future stock price scenarios based on the estimated models and compare these simulated paths with actual price data.
- Select the most accurate model for future predictions using Root Mean Squared Error.

The analysis is based on Google's stock prices (GOOG) spanning from January 1, 2019 to December 31, 2024. The following libraries are utilized to perform data acquisition, statistical analysis, modeling, and visualization:

```r
library(quantmod)              # Load financial data (Google stock prices)
library(xts)                   # Time series manipulation
library(rugarch)               # GARCH model estimation and forecasting
library(PerformanceAnalytics)  # Financial analysis (logarithmic returns)
library(tseries)               # Statistical tests for time series (ADF test)
library(fBasics)               # Basic financial statistics (dagoTest)
library(zoo)                   # Time series manipulation and rolling window analysis
```

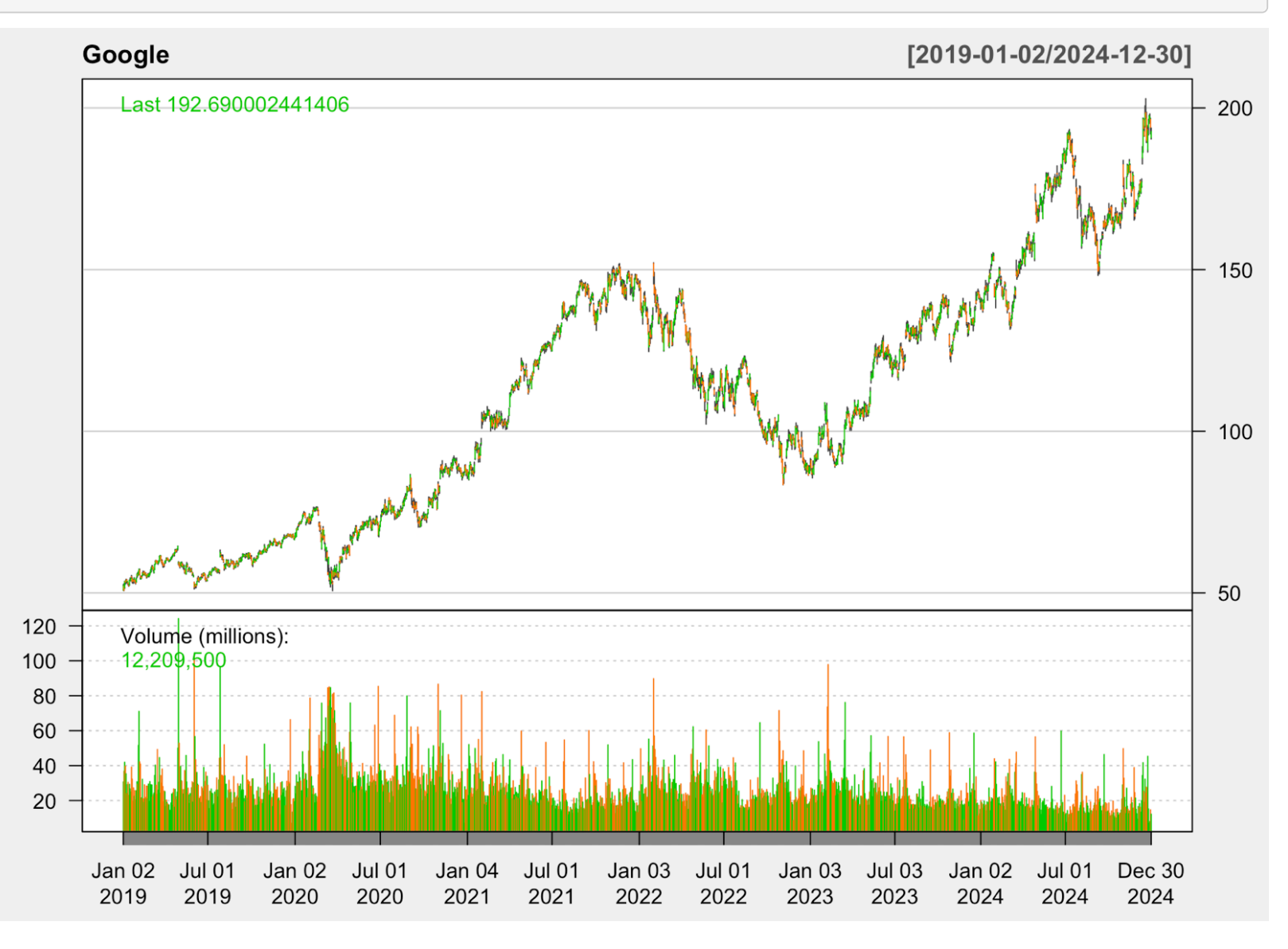
## 1. Data Loading & Preprocessing

I begin by loading Google stock data from Yahoo Finance and computing log returns. Before fitting any models, I check for stationarity, linearity, and normality to understand the properties of the time series. I download 'GOOG' stock data using the getSymbols function from the quantmod package. The data includes OHLC: Open, High, Low, and Close prices.

```r
G <- getSymbols("GOOG", from = "2019-01-01", to = "2024-12-31", auto.assign = FALSE)
```

This is the candlestick chart:

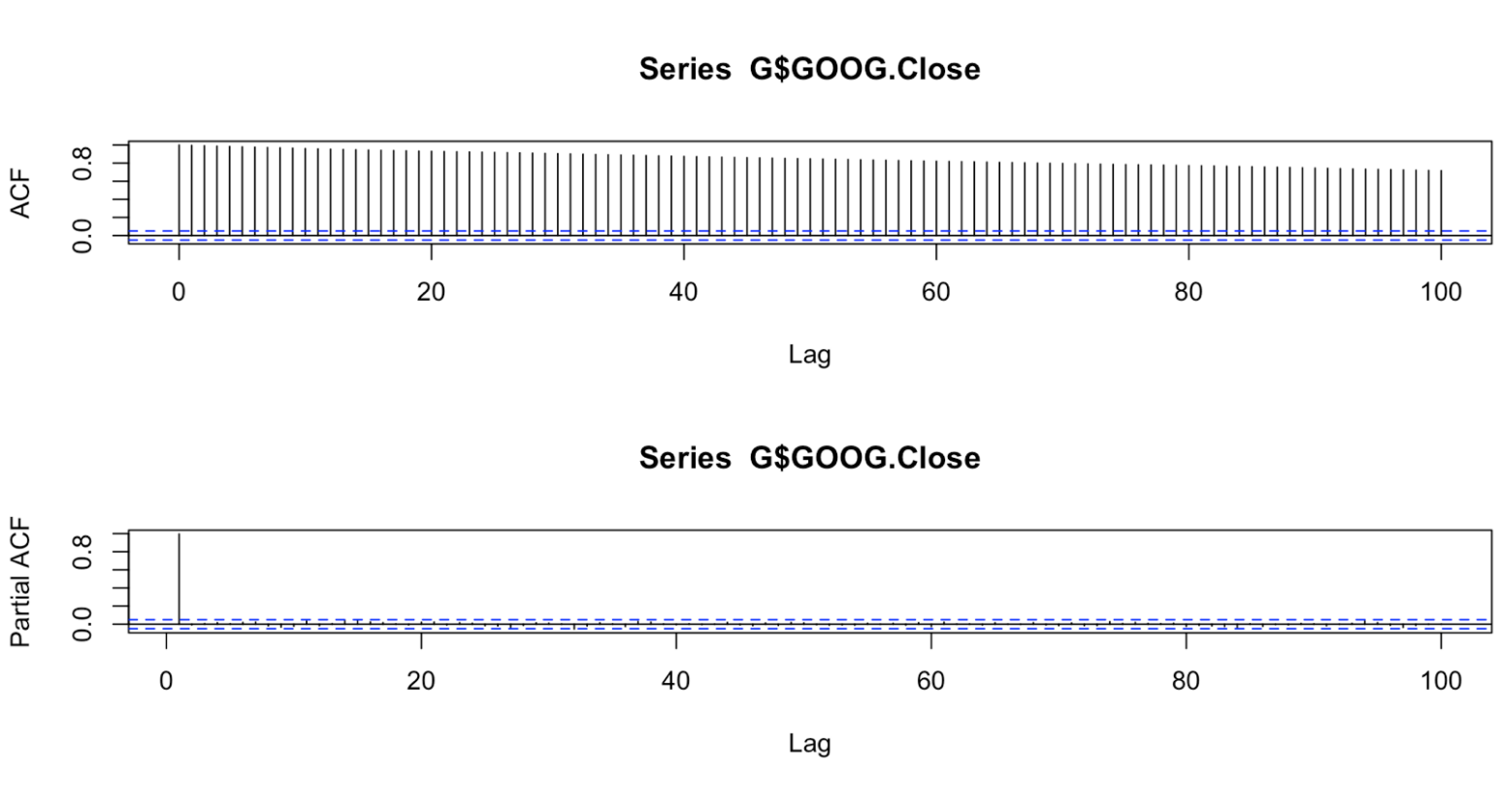
```r
chartSeries(name = "Google", G, theme = "white")
```



### Stationarity

The first step is to study the stationarity of the series at the levels. So, the correlograms of the ACF and PACF are typical of non-stationary series: as we can see, the ACF decays very slowly to zero and the PACF deviates significantly from the Bartlett bands at lag 1.

```r
par(mfcol=c(2,1))
acf(lag.max = 100, G$GOOG.Close)
pacf(lag.max = 100, G$GOOG.Close)
```



To confirm the presence of a unit root, I perform a stationarity test such as the Augmented Dickey-Fuller test, where the null hypothesis (Ho) is that the series has a unit root, meaning it is non-stationary.

```r
adf.test(G$GOOG.Close)
```

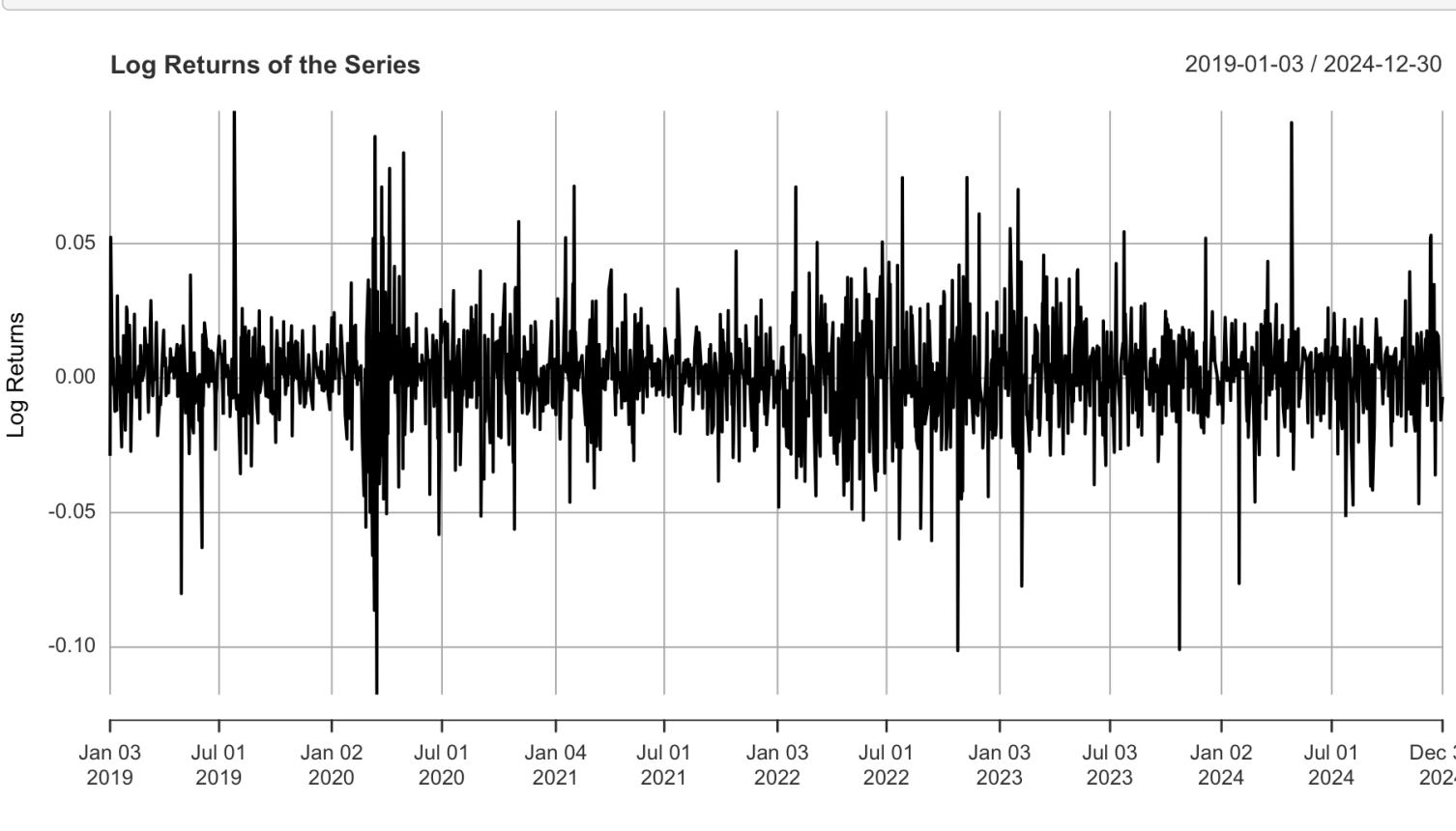
```
    Augmented Dickey-Fuller Test

data:  G$GOOG.Close
Dickey-Fuller = -1.8378, Lag order = 11, p-value = 0.647
alternative hypothesis: stationary
```

The test fails to reject the null hypothesis, suggesting that the series could have a unit root, so I can conclude that it may not be stationary. I proceed by differencing the series at the levels, so I compute the log returns to achieve stationarity.

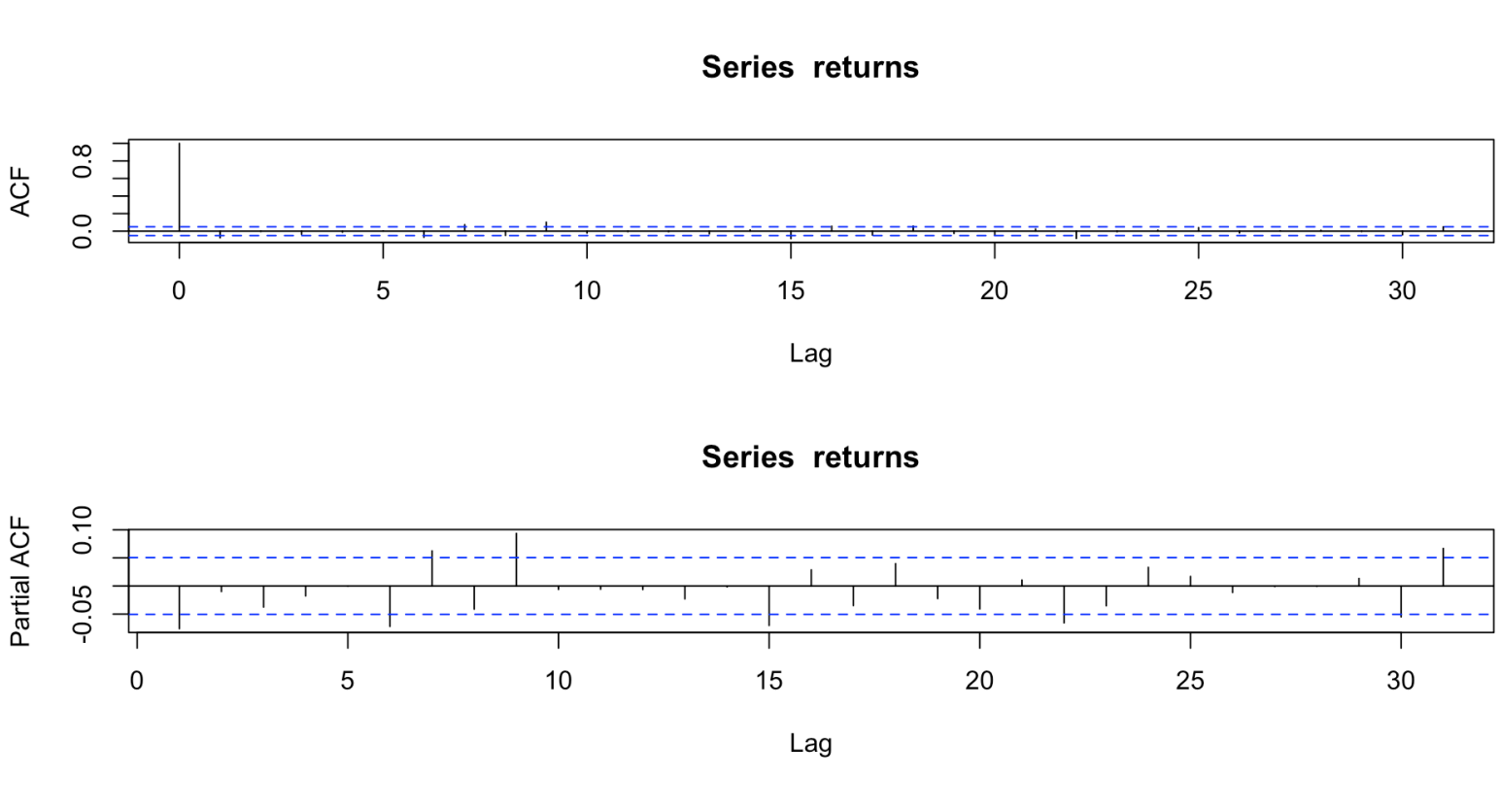
```r
returns <- CalculateReturns(G$GOOG.Close, method = "log") # Equivalent to: returns <- diff(log(G$GOOG.C
lose))[-1]
returns <- returns[-1] # Remove NA from first observation
```

The series of log returns is as follows:

```r
plot(returns, main = "Log Returns of the Series", xlab = "Time", ylab = "Log Returns")
```



I check the ACF and PACF of returns to confirm stationarity:



### Linearity

To test the linearity of the series, I convert it to a time series (ts) object and use the Teräsvirta test. The p-value is below the 5% threshold, suggesting that the series exhibits nonlinear behavior.

```r
terasvirta.test(ts(returns))
```

```
    Teraesvirta Neural Network Test

data:  ts(returns)
X-squared = 12.192, df = 2, p-value = 0.002252
```

This indicates that simple linear models might not be adequate for capturing the data dynamics. In other words, it may be necessary to consider more complex models, such as nonlinear models or GARCH models.

### Normality

To test the normality of the series, I apply the D'Agostino Normality Test. It strongly rejects the null hypothesis of normality (p-value < 2.2e-16). The series exhibits a more pronounced negative skewness (-3.1959) and extremely high kurtosis (12.3842), indicating significant asymmetry and heavy tails. This suggests that returns deviate substantially from a normal distribution, which is a common characteristic of financial time series, often requiring alternative distributional assumptions in modeling.

```r
dagoTest(returns)
```

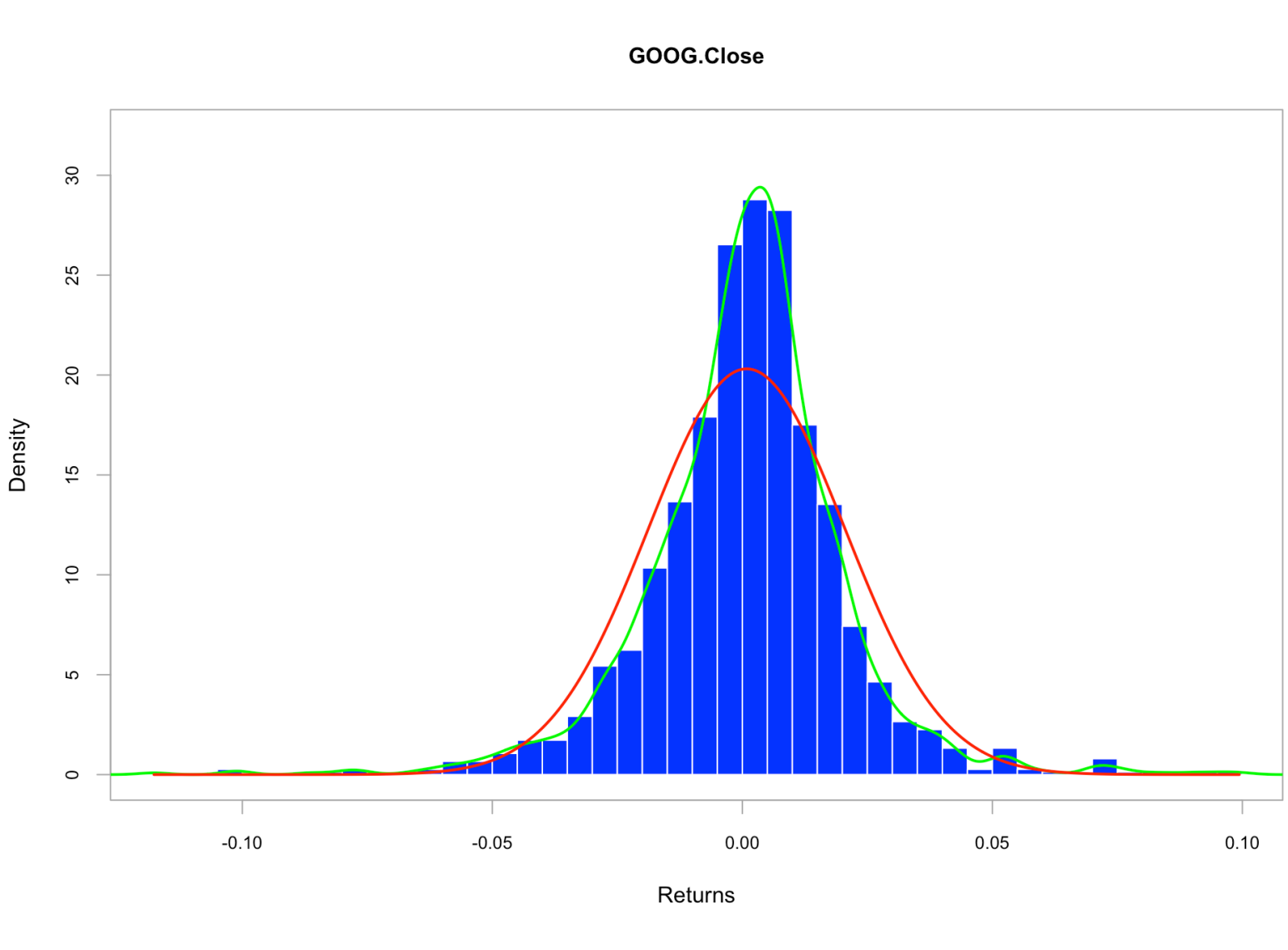
```

Title:
 D'Agostino Normality Test

Test Results:
  STATISTIC:
    Chi2 | Omnibus: 163.5833
    Z3  | Skewness: -3.1959
    Z4  | Kurtosis: 12.3842
  P VALUE:
    Omnibus  Test: < 2.2e-16
    Skewness Test: 0.001394
    Kurtosis Test: < 2.2e-16
```

```r
basicStats(returns) # Compute and display basic statistical metrics
```

```
              GOOG.Close
nobs         1508.000000
NAs             0.000000
Minimum        -0.117667
Maximum         0.099380
1. Quartile    -0.008692
3. Quartile     0.010849
Mean            0.000865
Median          0.001864
Sum             1.304230
SE Mean         0.000506
LCL Mean       -0.000127
UCL Mean        0.001857
Variance        0.000386
Stdev           0.019643
Skewness       -0.202397
Kurtosis        4.288964
```

I also plot a histogram with density and normal distribution overlays to visualize these characteristics:

```r
chart.Histogram(returns, methods = c('add.density', 'add.normal'), colorset = c('blue', 'green', 're
d'))
```



In this histogram of GOOG's daily returns, the red curve represents a normal distribution, while the green curve is a kernel density estimate. The actual return distribution appears more peaked and has heavier tails than the normal curve, indicating significant kurtosis and potential skewness. Because financial returns often deviate from normality, I included the skewed Student-t (sstd) specification in my GARCH models to better capture these heavy tails and asymmetric behavior.

## 2. Volatility Analysis

A thorough volatility analysis is essential before selecting the best forecasting model. This step helps assess market fluctuations and determine the most suitable approach for predicting future price movements.
I compute annualized volatility and rolling volatility trends to capture how market fluctuations evolve over time.

```r
annual_volatility <- sqrt(252) * sd(returns)
print(paste("Annualized volatility:", annual_volatility))
```

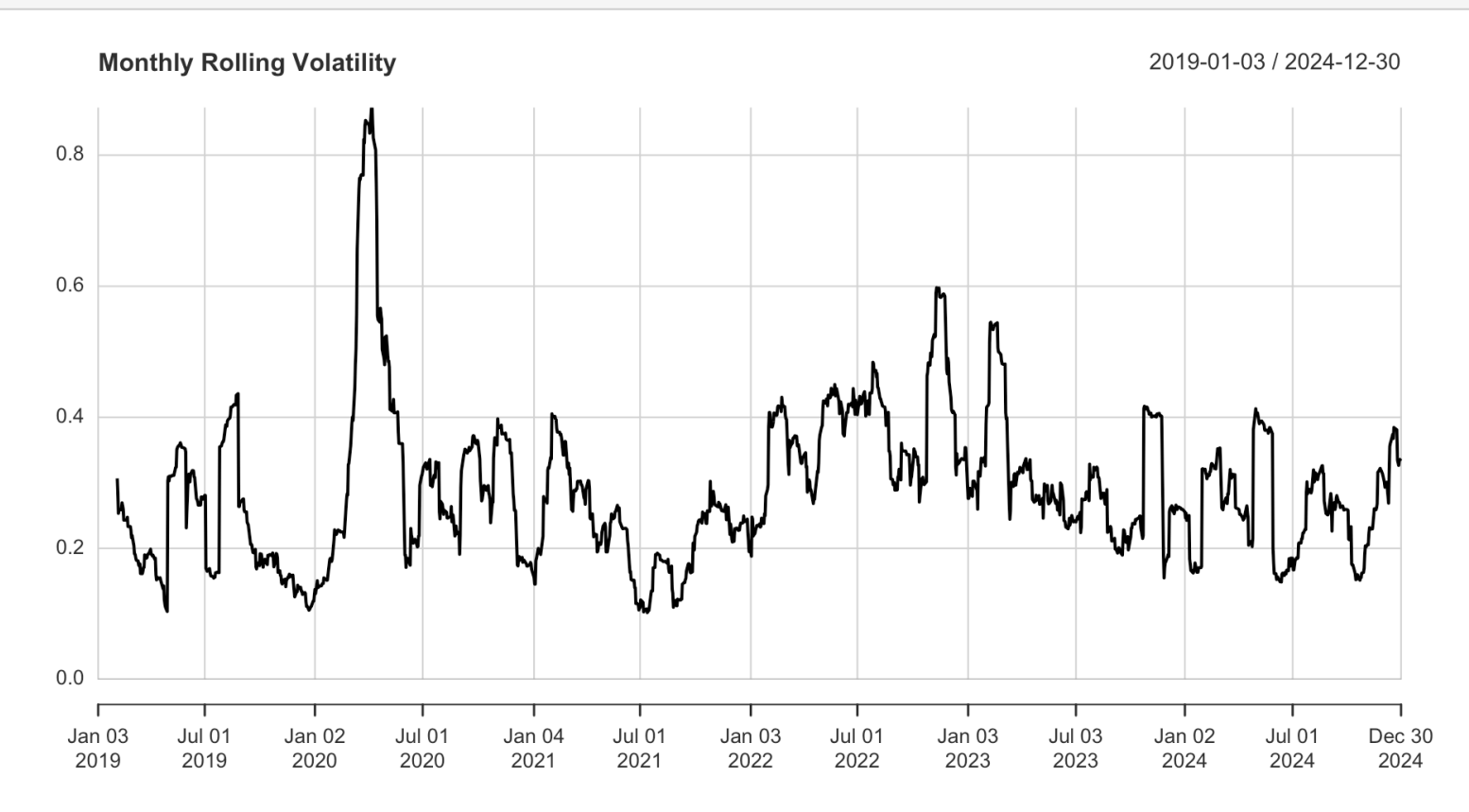
```
[1] "Annualized volatility: 0.311818212319738"
```

```r
years <- 2019:2024
volatility <- sapply(years, function(y) sqrt(252) * sd(returns[as.character(y)], na.rm = TRUE))
names(volatility) <- years
print(volatility)
```

```
     2019      2020      2021      2022      2023      2024 
0.2407846 0.3843914 0.2359527 0.3880051 0.3061957 0.2770294 
```

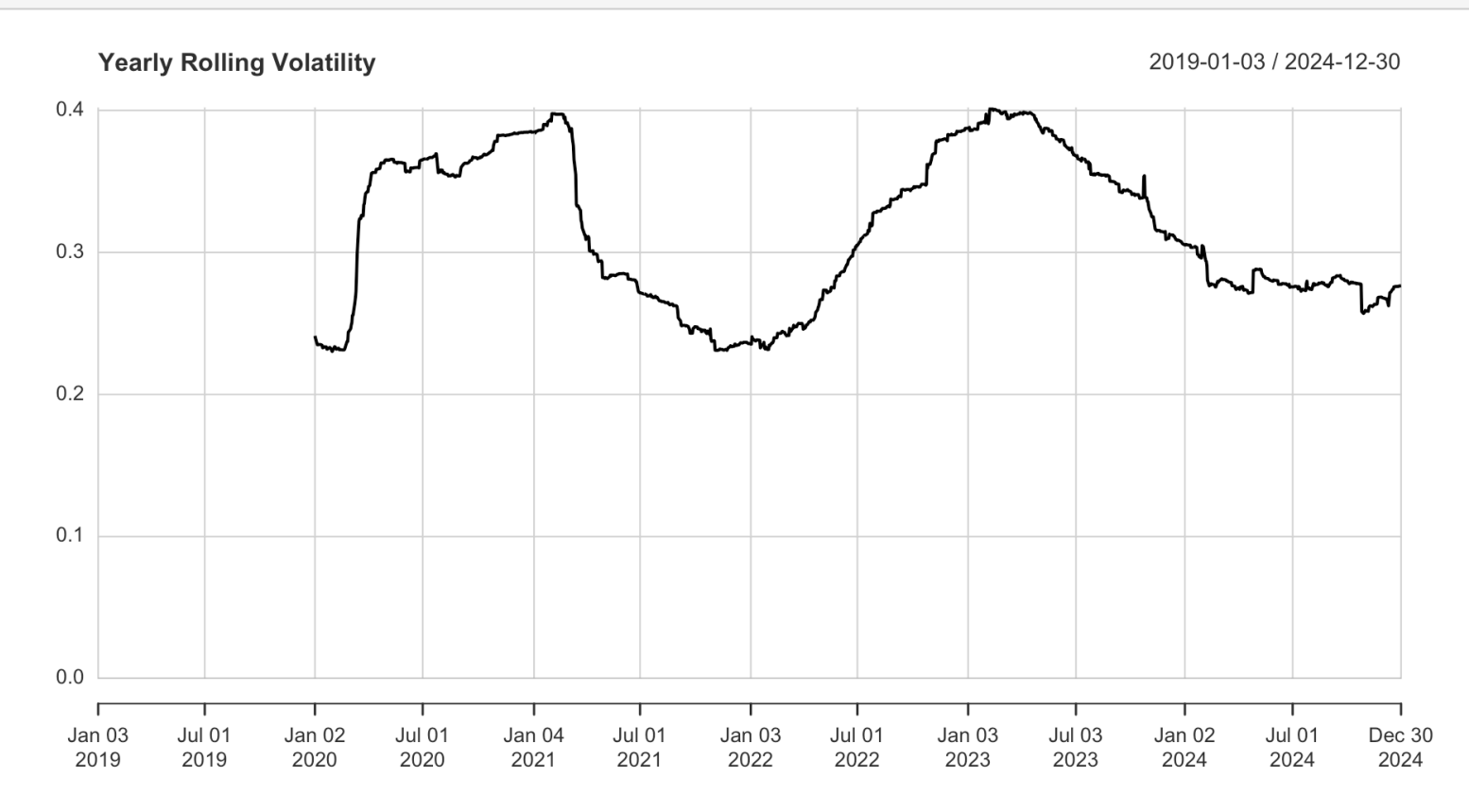
Monthly Rolling Volatility:

```r
chart.RollingPerformance(R = returns["2019::2024"], width = 22, FUN = 'sd.annualized', scale = 252, mai
n = "Monthly Rolling Volatility")
```



Yearly Rolling Volatility:

```r
chart.RollingPerformance(R = returns["2019::2024"], width = 252, FUN = 'sd.annualized', scale = 252, ma
in = "Yearly Rolling Volatility")
```



The computed annualized volatility is approximately 31.18%, indicating a relatively high level of fluctuations in Google's stock returns. This suggests that the stock experiences significant price swings over the year, which is typical for tech companies. High volatility implies greater risk but also potential opportunities for traders.

## 3. Rolling Window

In this section, I define and estimate different GARCH-type models to capture the volatility dynamics of Google's stock returns.
I consider standardGARCH, gjrGARCH and eGARCH models with both normal and skewed Student-t distributions. The rolling window approach (one-year window) is used to capture time-varying volatility patterns and generate short-term forecasts (approximately two months ahead).

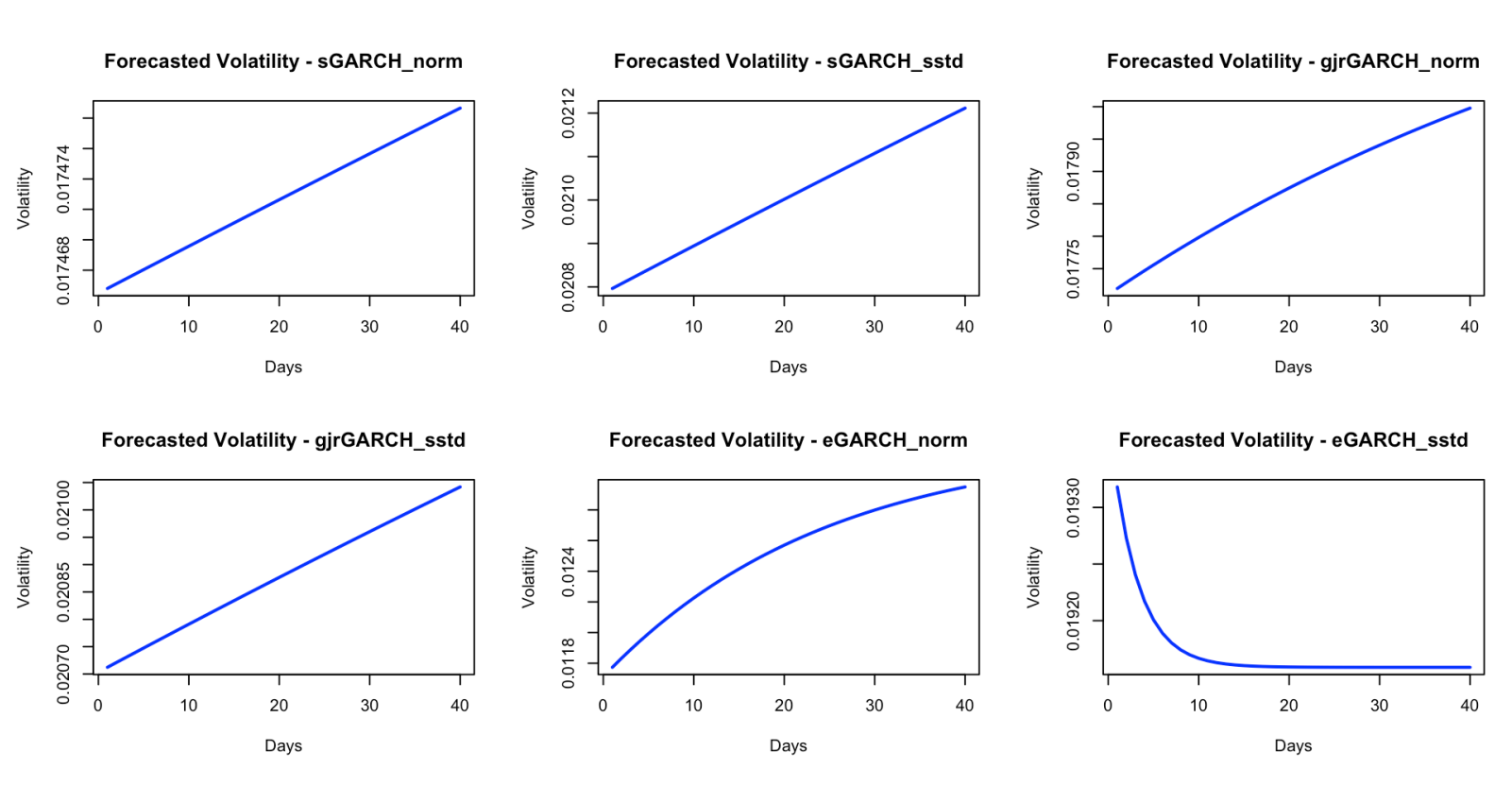
```r
# Define models to be estimated
models_list <- list(

  # Standard GARCH model with normal distribution
  sGARCH_norm = ugarchspec(mean.model = list(armaOrder = c(0,0)),
                           variance.model = list(model = "sGARCH"),
                           distribution.model = "norm"),

  # Standard GARCH model with skewed Student-t distribution
  sGARCH_sstd = ugarchspec(mean.model = list(armaOrder = c(0,0)),
                           variance.model = list(model = "sGARCH"),
                           distribution.model = "sstd"),

  # GJR-GARCH model (asymmetric GARCH) with normal distribution
  gjrGARCH_norm = ugarchspec(mean.model = list(armaOrder = c(0,0)),
                             variance.model = list(model = "gjrGARCH"),
                             distribution.model = "norm"),

  # GJR-GARCH model with skewed Student-t distribution
  gjrGARCH_sstd = ugarchspec(mean.model = list(armaOrder = c(0,0)),
                             variance.model = list(model = "gjrGARCH"),
                             distribution.model = "sstd"),

  # Exponential GARCH model with normal distribution
  eGARCH_norm = ugarchspec(mean.model = list(armaOrder = c(0,0)),
                           variance.model = list(model = "eGARCH"),
                           distribution.model = "norm"),

  # Exponential GARCH model with skewed Student-t distribution
  eGARCH_sstd = ugarchspec(mean.model = list(armaOrder = c(0,0)),
                                variance.model = list(model = "eGARCH"),
                                distribution.model = "sstd")
)
```

So, I implement a rolling window over six years estimation procedure to dynamically assess the volatility of Google stock returns over time. I aim to capture time-varying volatility patterns and improve forecasting accuracy. This approach allows me to generate short-term volatility forecasts while accounting for changes in market conditions.

```r
# Rolling window settings
window_size <- 251  # Approx. 1 year of trading days (assuming ~252 trading days per year)
forecast_horizon <- 40  # Approx. 2 months of trading days
n_obs <- length(returns) # Total number of observations in the dataset
results <- list() # Store results for each model

# Perform rolling estimation for each model
for (model_name in names(models_list)) {
  cat("Estimating rolling window for model:", model_name, "\n") # Print current model being estimated

  spec <- models_list[[model_name]]  # Select the model specification from the predefined list
  model_results <- list() # Store fitted models for different periods
  forecasts <- matrix(NA, nrow = 6, ncol = forecast_horizon)  # Initialize matrix to store volatility f
orecasts

  for (year in 0:5) {  # Iterate over six rolling periods (approx. six years)
    start_index <- (year * window_size) + 1 # Define rolling window start index
    end_index <- start_index + window_size - 1 # Define rolling window end index
    if (end_index > n_obs) break # Stop if the end index exceeds available data

    window_data <- returns[start_index:end_index]  # Extract data for the rolling window

    fit <- ugarchfit(spec = spec, data = window_data)  # Fit the model to the window data

    if (fit@fit$convergence == 0) {  # Check if the model successfully converged
      model_results[[year + 1]] <- fit  # Store fitted model for the given year

      forecast <- ugarchforecast(fit, n.ahead = forecast_horizon) # Generate volatility forecasts
      forecasts[year + 1, ] <- as.numeric(sigma(forecast))  # Store predicted volatility values
    } else {
      cat("Model", model_name, "for year", 2019 + year, "did not converge\n") # Warning if the model fa
ils to converge
    }
  }
  # Store results for the current model, including fitted models and forecasts
  results[[model_name]] <- list(models = model_results, forecasts = forecasts)
}
```

```
Estimating rolling window for model: sGARCH_norm 
Estimating rolling window for model: sGARCH_sstd 
Estimating rolling window for model: gjrGARCH_norm 
Estimating rolling window for model: gjrGARCH_sstd 
Estimating rolling window for model: eGARCH_norm 
Estimating rolling window for model: eGARCH_sstd 
```

Plot forecasted volatility for the last rolling window:



In this figure, five of the six models (sGARCH_norm, sGARCH_sstd, gjrGARCH_norm, gjrGARCH_sstd and eGARCH_norm) forecast a gradual rise in volatility, whereas the eGARCH_sstd model exhibits a rapid decline. This discrepancy may stem from the specific parameter estimates obtained by the eGARCH_sstd specification. Because eGARCH can capture asymmetric effects differently, especially when combined with a skewed Student-t distribution, its forecast can diverge if recent market conditions lead the model to project a swift reversion to lower volatility. Such variations highlight the importance of comparing multiple GARCH specifications, as each model can respond uniquely to the same historical data and produce distinct volatility trajectories.
This figure provides insight into how different models forecast future volatility. The next step is to determine which forecasting model best fits the data by evaluating its predictive accuracy.

## 4. Price Simulation

Now I define key parameters for the Google stock prices simulation of January - February 2025. I set the number of simulation paths and forecast days, extract the last observed price, and initialize lists to store simulation results and RMSE values.

```r
m.sim <- 100  # Number of simulations per model
n.sim <- 38   # Number of forecast days (Jan-Feb 2025)
last_price <- as.numeric(last(G$GOOG.Close))  # Last observed stock price
rmse_results <- list() # Store RMSE values for each model
model_names <- names(results) # Extract model names
simulated_prices_list <- list() # Initialize storage for simulated prices
```

I then simulate future stock prices using the last successfully estimated GARCH model for each specification. The code checks that each model has valid estimations (skipping those with missing or non-converged results) before fixing the estimated parameters. It then simulates future returns with "ugarchpath" and converts these returns into prices for further analysis.

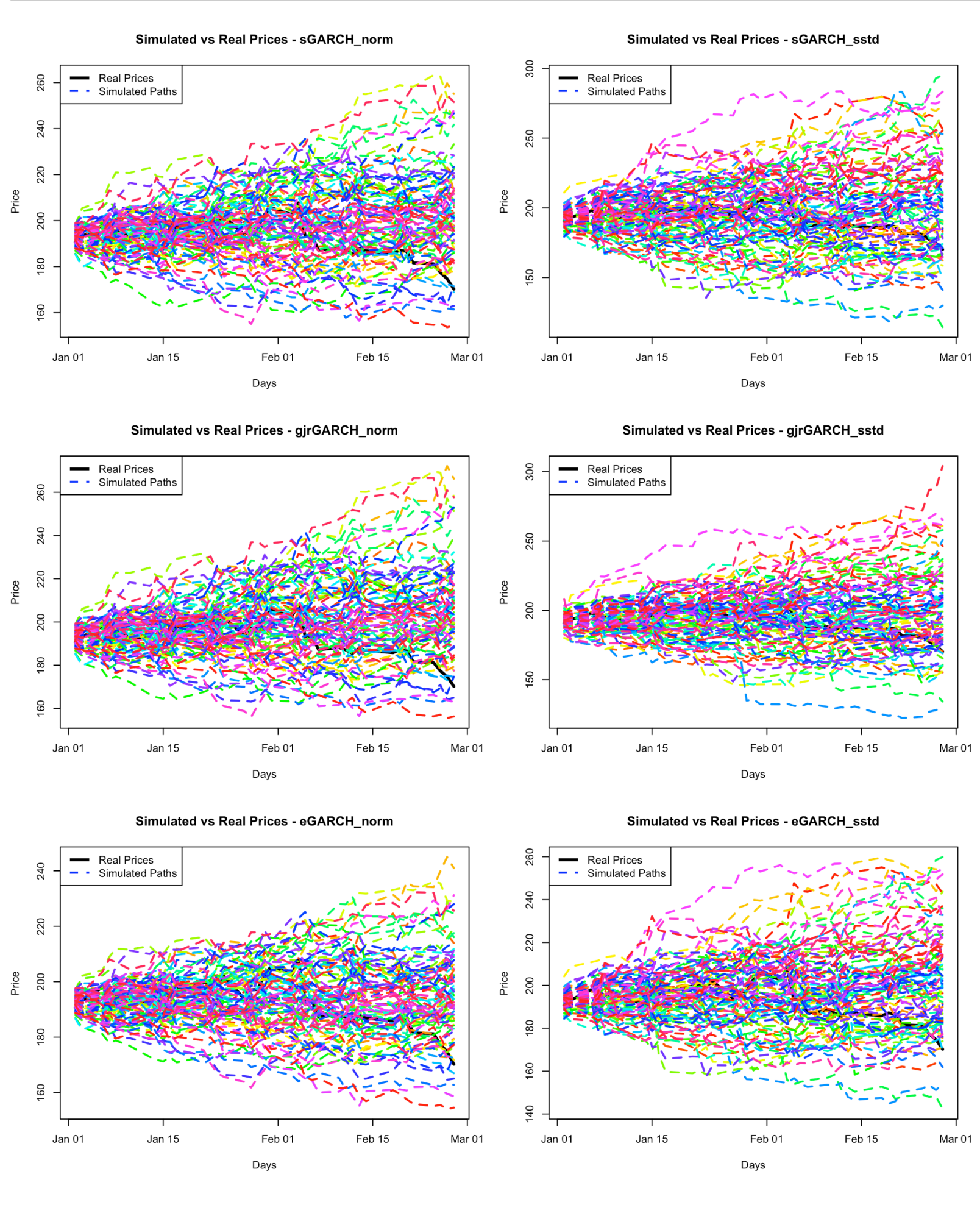
```r
for (model_name in model_names) {
  cat("Simulating future prices for model:", model_name, "\n")

  # Get the last estimated model (from the last rolling window)
  last_index <- length(results[[model_name]]$models)

  if (last_index == 0) {
    cat("Skipping model", model_name, "due to missing estimations\n")
    next
  }

  last_model <- results[[model_name]]$models[[last_index]]

  # Check if the model successfully converged
  if (is.null(last_model)) {
    cat("Skipping", model_name, "as the last model did not converge.\n")
    next
  }

  # Create a new ugarchspec with fixed parameters from the last estimated model
  sfinal <- models_list[[model_name]]
  fixed_params <- as.list(coef(last_model))

  sfinal_fixed <- ugarchspec(
    mean.model = list(armaOrder = c(0,0)),
    variance.model = list(model = last_model@model$modeldesc$vmodel),
    distribution.model = last_model@model$modeldesc$distribution
  )

  setfixed(sfinal_fixed) <- fixed_params  # Fix the parameters

  # Simulate future returns
  sim <- ugarchpath(spec = sfinal_fixed, m.sim = m.sim, n.sim = n.sim, rseed = 123)

  # Convert simulated returns to prices
  simulated_returns <- fitted(sim)
  simulated_prices <- last_price * exp(apply(simulated_returns, 2, cumsum))

  # Store simulated prices
  simulated_prices_list[[model_name]] <- simulated_prices
}
```

```
Simulating future prices for model: sGARCH_norm 
Simulating future prices for model: sGARCH_sstd 
Simulating future prices for model: gjrGARCH_norm 
Simulating future prices for model: gjrGARCH_sstd 
Simulating future prices for model: eGARCH_norm 
Simulating future prices for model: eGARCH_sstd 
```

## 5. Simulated vs Real Prices

In this section, I compare simulated price paths with actual prices for Jan-Feb 2025. The simulated paths help me assess the model's predictive performance.
Before plotting the simulated versus real prices, I verify that each model's simulation data is not null. This check is essential because some models may not produce valid simulation outputs, due to convergence issues or other errors, and attempting to plot a null object would result in errors. By ensuring that only models with valid simulation data are processed, I can safely generate the comparison plots.

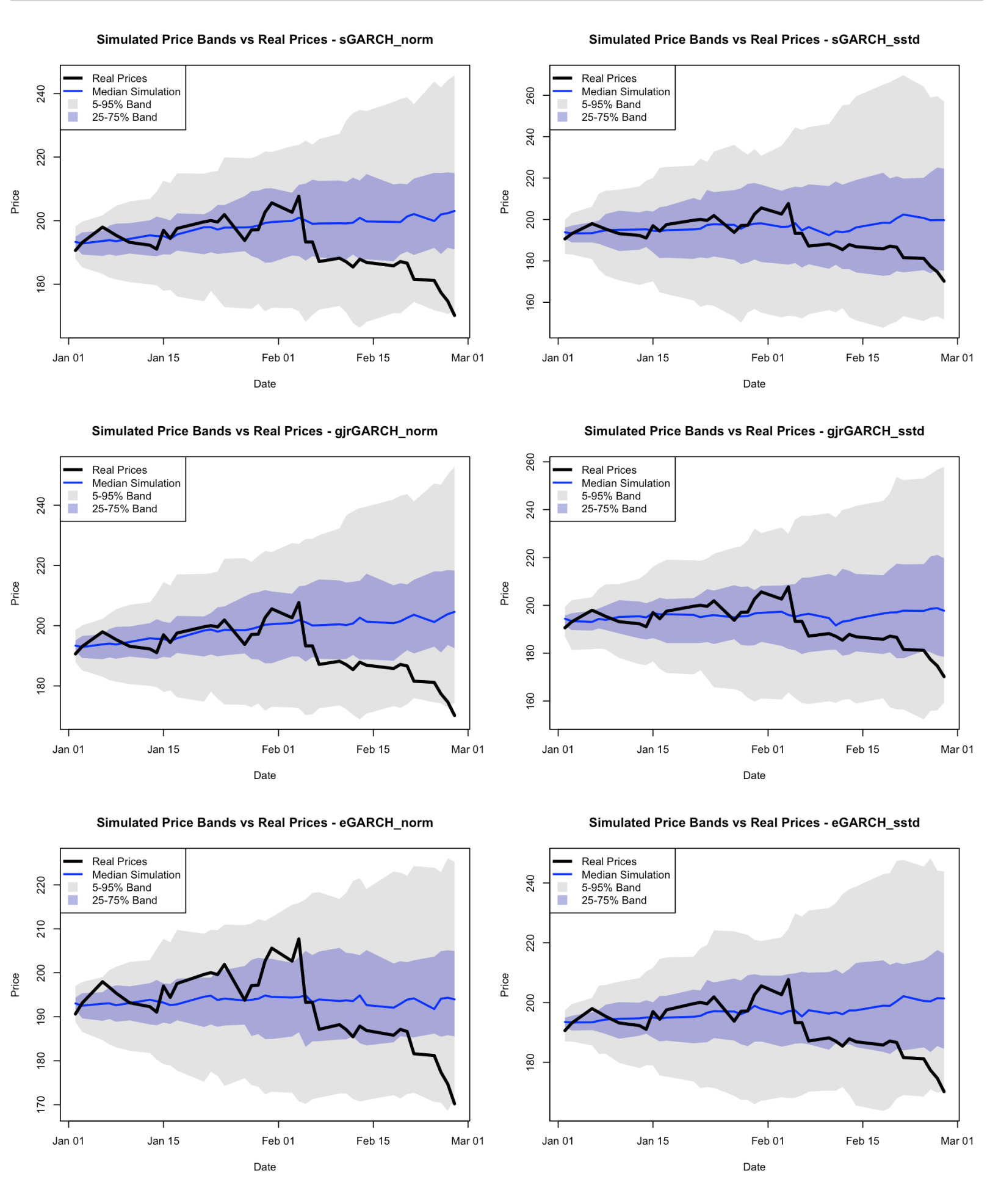
```r
par(mfrow = c(3, 2))  # Arrange plots in a grid

# Load actual prices for Jan-Feb 2025
real_prices <- getSymbols("GOOG", from = "2025-01-01", to = "2025-02-28", auto.assign = FALSE)$GOOG.Clo
se

# Comparing simulated prices with real stock prices.
for (model_name in model_names) {
  if (!is.null(simulated_prices_list[[model_name]])) {
    simulated_prices <- simulated_prices_list[[model_name]]

    plot(index(real_prices), real_prices, type = "l", col = "black", lwd = 3,
         ylim = range(simulated_prices, real_prices), main = paste("Simulated vs Real Prices -", model_
name),
         ylab = "Price", xlab = "Days")

    colors <- rainbow(m.sim)
    for (i in 1:m.sim) {
      lines(index(real_prices)[1:nrow(simulated_prices)], simulated_prices[, i], col = colors[i], lwd =
2, lty = 2)
    }

    legend("topleft", legend = c("Real Prices", "Simulated Paths"),
           col = c("black", "blue"), lwd = c(3, 2), lty = c(1, 2))
  }
}
```



## 6. Quantile Bands

Next, I calculate the quantiles (5th, 25th, 50th, 75th, and 95th percentiles) for each forecast day across the simulated paths and visualize them as shaded bands.
These bands illustrate where the majority of simulations concentrate and help me compare the forecast against actual prices.

```r
# Assume that:
# - simulated_prices_list is a list of matrices, where each matrix has:
#     rows = forecast days
#     columns = simulations (m.sim)
# - real_prices is an xts series containing the actual prices for the period of interest

par(mfrow = c(3, 2))  # Arrange plots in a grid

# For each model, I compute the quantiles for each day and plot the graphs:
for(model_name in names(simulated_prices_list)) {

  simulated_prices <- simulated_prices_list[[model_name]]

  # Compute the quantiles for each day (each row) across the simulations
  # This step uses the 'apply' function to work over the rows (MARGIN = 1) of the matrix
  # I compute the 5th, 25th, 50th, 75th, and 95th percentiles
  quantiles <- t(apply(simulated_prices, 1, function(x) {
    quantile(x, probs = c(0.05, 0.25, 0.5, 0.75, 0.95), na.rm = TRUE)
  }))

  # Get the forecast dates. I assume that these correspond to the first n rows of real_prices.
  forecast_dates <- index(real_prices)[1:nrow(simulated_prices)]

  # Set the y-axis range based on both the quantiles and the actual real prices
  y_range <- range(c(quantiles, coredata(real_prices)[1:nrow(simulated_prices)]), na.rm = TRUE)

  # Plot the real prices. The real prices are plotted as a solid black line
  plot(forecast_dates, coredata(real_prices)[1:nrow(simulated_prices)],
       type = "l", lwd = 3, col = "black",
       main = paste("Simulated Price Bands vs Real Prices -", model_name),
       xlab = "Date", ylab = "Price", ylim = y_range)

  # Draw a polygon to represent the area between the 5th and 95th percentiles (this is the low-intensit
y zone)
  # This polygon creates a shaded band indicating where 90% of the simulations lie
  polygon(c(forecast_dates, rev(forecast_dates)),
          c(quantiles[, "5%"], rev(quantiles[, "95%"])),
          col = rgb(0.8, 0.8, 0.8, alpha = 0.5), border = NA)

  # Draw a polygon to represent the area between the 25th and 75th percentiles (this is the high-concen
tration zone)
  # This band shows the interquartile range, highlighting the central 50% of the simulations
  polygon(c(forecast_dates, rev(forecast_dates)),
          c(quantiles[, "25%"], rev(quantiles[, "75%"])),
          col = rgb(0.4, 0.4, 0.8, alpha = 0.5), border = NA)

  # Plot the median simulation line (50th percentile) in blue.
  lines(forecast_dates, quantiles[, "50%"], col = "blue", lwd = 2)

  # Re-plot the real prices line to ensure it is clearly visible on top of the polygons
  lines(forecast_dates, coredata(real_prices)[1:nrow(simulated_prices)], col = "black", lwd = 3)

  # Add a legend to explain the different components of the plot
  legend("topleft",
         legend = c("Real Prices", "Median Simulation", "5-95% Band", "25-75% Band"),
         col = c("black", "blue", rgb(0.8, 0.8, 0.8, alpha = 0.5), rgb(0.4, 0.4, 0.8, alpha = 0.5)),
         lwd = c(3, 2, NA, NA),
         pch = c(NA, NA, 15, 15),
         pt.cex = 2)
}
```



In these plots, I compare the simulated price bands for each GARCH model with the actual prices from January 1 to March 1, 2025.

The black line represents the real observed prices, while the gray and blue shaded regions indicate the 5–95% and 25–75% simulation intervals, respectively. Notably, the real prices experienced a sudden drop in mid-February, which was only partially captured by the simulation bands. This sharp decline could be linked to a major market event, possibly related to “DeepSeek” or another external factor, that significantly impacted investor sentiment. Such abrupt shocks underscore the challenge of predicting extreme movements with volatility models, which rely heavily on historical patterns and may not fully account for unexpected market catalysts.

# 7. Model Validation: RMSE

I evaluate model performance using the Root Mean Squared Error (RMSE) to compare simulated prices with actual prices. Before proceeding with the RMSE calculation, I note that the Root Mean Squared Error is a widely used metric for evaluating predictive accuracy because it provides a single measure that penalizes large deviations between the predicted and actual values. By squaring the residuals, the RMSE places extra emphasis on significant errors, ensuring that models producing large prediction gaps are penalized more heavily. This makes the RMSE a straightforward and effective criterion for comparing the overall performance of different forecasting models.

```r
# Evaluating model performance using Root Mean Squared Error
for (model_name in model_names) {
  if (!is.null(simulated_prices_list[[model_name]])) {
    simulated_prices <- simulated_prices_list[[model_name]]

    # Compute RMSE for each simulated scenario
    rmse_values <- sapply(1:m.sim, function(i) sqrt(mean((real_prices - simulated_prices[, i])^2, na.rm
= TRUE)))
    rmse_mean <- mean(rmse_values)
    rmse_results[[model_name]] <- rmse_mean
    cat("RMSE for model", model_name, ":", rmse_mean, "\n")
  }
}
```

```
RMSE for model sGARCH_norm : 17.66665
RMSE for model sGARCH_sstd : 25.17159
RMSE for model gjrGARCH_norm : 18.72114
RMSE for model gjrGARCH_sstd : 20.73697
RMSE for model eGARCH_norm : 13.78883
RMSE for model eGARCH_sstd : 18.5666
```

Finally, I identify the best model based on the lowest RMSE value. The best model is chosen as the one with the minimum RMSE, indicating the closest fit to actual prices.

```r
# Identifying the best model using RMSE performance
best_model <- names(which.min(rmse_results))
cat("The best model based on RMSE is:", best_model, "with RMSE ->", rmse_results[[best_model]], "\n")
```

```
The best model based on RMSE is: eGARCH_norm with RMSE -> 13.78883
```

# Conclusion

In conclusion, this project has demonstrated the process of analyzing and forecasting volatility in a financial time series using various GARCH models. By comparing standard and more flexible specifications (e.g., skewed Student-t), rolling-window estimation, and simulation-based techniques, I have illustrated how model choice can significantly influence both volatility forecasts and price predictions. The results show that some models capture the dynamics better than others, as evidenced by lower RMSE values.

## Key February 2025 Events

- Google’s Q4 2024 Earnings Report: Google likely released its Q4 2024 earnings report in early February. A weaker than expected revenue or profit in its core businesses (advertising, cloud, AI, or hardware) could have caused a negative stock reaction. Conversely, strong earnings could have led to a temporary rally.
- U.S. Trade Policies: on February 1, 2025, President Trump announced 25% tariffs on imports from Canada and Mexico and 10% on Chinese goods. While initially suspended, these tariffs were later enforced, prompting China to retaliate with 15% tariffs on selected U.S. products.
- AI: in February 2025, competitive pressures in AI intensified, impacting investor confidence in Google. Microsoft/OpenAI continued integrating advanced AI models into Bing, challenging Google’s dominance in search. Meanwhile, Apple and Amazon expanded their presence in cloud computing and AI-powered advertising, increasing market competition. Additionally, DeepSeek, a rising AI competitor, introduced new large language models (LLMs) with enhanced efficiency, posing a potential threat to Google’s AI leadership. These developments may have influenced market sentiment and contributed to fluctuations in Google’s stock price.